

Mini-project 2 in Advanced Machine Learning (02460)

by Mads ALKJAERSIG (s203372), Rémi BERTHELOT (s254144) & Pierre HOLLEBEQUE (s254136) on 05 March 2026

Part A: pullback geodesics

In this project, we consider a subset of MNIST containing 3 classes and 2048 observations. In the first part, we employ a VAE with a standard Gaussian prior and a Gaussian likelihood. The latent space is two-dimensional to ease plotting. The goal is to compute geodesics on the latent space under the pullback metric associated with the mean of the Gaussian decoder.

To compute these geodesics, an energy-minimizing approach is used based on the following discrete approximation: $\mathcal{E}(c) = \sum_{i=0}^{N-1} \|f(c(t_i)) - f(c(t_{i+1}))\|^2$ where c is a discretized curve in the latent space modeled with both piecewise linear and polynomial approaches.

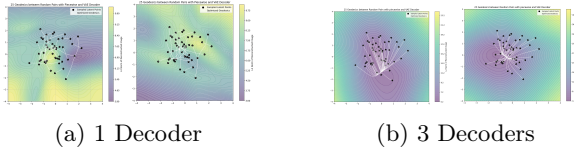


Figure 1: Geodesics between 25 random latent pairs. The background represents the L_2 norm of the reconstructed image. Left: Standard VAE showing geometric inconsistency. Right: Ensemble VAE with $M = 3$ decoders.

Figure 1a show that geodesics deviate from Euclidean straight lines as expected. However, their specific shapes vary across models. This indicates that the decoder mapping f is highly sensitive to training stochasticity, resulting in inconsistent pullback metric landscapes and limited reliability.

Part B: Ensemble VAE geometry

The learned decoder for a VAE model is not deterministic - the resulting decoder weights is derived from the initialized parameters through SGD. From a Bayesian perspective, the most accurate representation of the model's predictive uncertainty would be obtained by marginalizing over the distribution of M possible decoders via the expectation $\mathbb{E}_{q(\theta)}[f_{\theta}(z)]$: it is an ensemble VAE. The resulting manifold will better account for the model's uncertainty and become more robust in the process. This can be applied to our energy-minimization function as such, $\mathcal{E}(c) \approx \sum_{i=0}^N \mathbb{E}_{l,k} [\|f_l(c(t_i)) - f_k(c(t_{i+1}))\|^2]$, where f_l, f_k are decoders drawn uniformly and the expectation is determined through Monte Carlo. Figure 1b shows that ensemble VAE have more similar geodesics structures where paths all pass through an attraction zone.

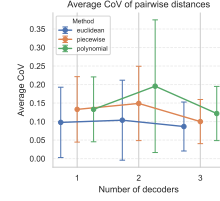


Figure 2: CoV across multiple training runs for different numbers of decoders - plotted against Euclidean distance.

To assess the reliability of different distance measures in the latent space, we use the coefficient of variation (CoV), a unit-less metric that quantifies variability relative to the mean. By training 10 separate but similar VAE models, we evaluate the consistency of geodesic paths across multiple runs. For each model, we compute distances for 10 randomly selected pairs using three methods: Euclidean distance, piecewise geodesics (with 20 segments), and polynomial geodesics (4th-order approximation). The CoV for each pair is then calculated as the ratio of the standard deviation to the mean of these distances across all models:

$$\text{CoV}_{ij} = \frac{\sigma(\{d_{ij}^{(1)}, \dots, d_{ij}^{(M)}\})}{\mu(\{d_{ij}^{(1)}, \dots, d_{ij}^{(M)}\})}, \text{ where } M = 10.$$

Figure 2 tracks the evolution of CoV as the number of decoders increases from 1 to 3. Theoretically, adding more decoders should reduce variability, as the ensemble better captures the expected decoder manifold. The results confirm this: the average CoV decreases across all three distance methods, with the Euclidean and piecewise geodesic approaches showing the most pronounced improvement. The polynomial method, however, exhibits higher variability, particularly with only two decoders. Overall, the reduction in both mean and variance of CoV supports the hypothesis that ensemble VAEs yield more consistent and reliable latent space geometries.

This study explores the geometric properties of Variational Autoencoder (VAE) latent spaces, focusing on the reliability of geodesics computed under the pullback metric. Standard VAEs exhibit high sensitivity to training stochasticity, resulting in inconsistent geodesic paths across independent runs, as shown in Figure 1. An ensemble VAE approximates the expected decoder manifold by marginalizing over multiple decoders, thereby it accounts for more model uncertainty. Using the coefficient of variation (CoV), we demonstrate that ensemble VAEs with more decoders yield more reliable and consistent geodesic distances.

Code snippets

Part A

Energy Computation:

```

1 def calculate_energy(self, decoder=None):
2     curve_points = self.get_full_curve_points()
3     reconstructed_images = decoder(curve_points).mean
4     segment_image_diffs = reconstructed_images[1:] - reconstructed_images[:-1]
5     squared_image_distances = torch.sum(segment_image_diffs**2, dim=list(range(1,
6     segment_image_diffs.ndim)))
7     energy = torch.sum(squared_image_distances)
8     return energy

```

Computing geodesics (details of the piecewise and polynomial classes are not included here due to space constraints, but can be found in the associated code):

```

1 def minimize_energy(self, num_iterations=100):
2     for i in range(num_iterations):
3         self.optimizer.zero_grad()
4         energy = self.curve_method.calculate_energy(self.decoder)
5         energy.backward()
6         self.optimizer.step()
7     return self.curve_method.get_full_curve_points()

```

Part B

Computing CoV matrix

```

1 def compute_cov_matrix(D):
2     mean = D.mean(axis=0)
3     std = D.std(axis=0)
4     cov = std/mean
5
6     return cov

```

Generate Distance Tensor

```

1 for m in range(num_models_per_D):
2     model = models[m]
3     for k in range(num_latent_points):
4         # Get the image pair (same for all models)
5         image_pair = image_pairs[k] # Shape: (2, 1, 28, 28)
6
7         # Encode through this model and extract latent means
8         x1, x2 = encode_images_to_latent_means(image_pair, model, device)
9
10        # Compute geodesic distance between latent means for this model
11        meta = compute_geodesic(
12            x1, x2, model, curve_method_str=curve_method_str,
13            number_parameters_geodesic=number_parameters_geodesic,
14            num_iter=num_iter, lr=lr,
15            device=device, return_metadata=True
16        )
17
18        dist_mat[m, k] = meta["distance"]

```

Declaration of use of generative AI

I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Gemini cli
- DeepL

Describe how the tools were used:

What did you use the tool(s) for? We have used Gemini to tailor the the code implementation and DeepL to assess the report grammar.

At what stage(s) of the process did you use the tool(s)? Gemini cli tool was used to verify our implementations of DDPM and VAE. It was also used to have a first draft of the different plots wanted for the report, and to document code. On the other hand, as none english native, DeepL was used to validate and rephrase parts of the report.

How did you use or incorporate the generated output? The generated output propose text that is embedded as proposed modifications inside modern idle like VSCode. Based on the proposed modification, we accept or refuse every parts to make sure the changes are tailored to our thoughts.